

A NOVEL METHODOLOGY FOR OPTIMIZING BLOOD DONATION AND BLOOD INVENTORY MANAGEMENT

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

ABHIJIT MODAK	22050440002
AANCHAL SINGH	22050440001
BHAVYA RATHOR	22050440032

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled **A NOVEL METHODOLOGY FOR OPTIMIZING BLOOD DONATION AND BLOOD INVENTORY MANAGEMENT** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: 12/06/2026

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA

(Head of Department)

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled **A NOVEL METHODOLOGY FOR OPTIMIZING BLOOD DONATION AND BLOOD INVENTORY MANAGEMENT** is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

AANCHAL SINGH

22050440001

ABSTRACT

Blood is one of the most critical resources in healthcare systems worldwide. Timely availability of blood can significantly improve patient outcomes during surgeries, accidents, emergencies, and medical treatments. However, traditional blood donation and inventory management systems often rely on manual processes, fragmented records, and inefficient communication mechanisms, leading to delays in locating suitable donors and managing blood inventories effectively.

This project, titled "**A Novel Methodology for Optimizing Blood Donation and Blood Inventory Management**," presents a comprehensive web-based solution developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The proposed system integrates donor registration, blood request management, hospital coordination, blood inventory tracking, and secure user authentication into a centralized platform.

The system employs Redux for efficient state management, JWT-based authentication for secure access control, and Nodemailer for automated email notifications. The platform supports multiple user roles, including administrators, donors, recipients, and hospitals/blood banks, enabling seamless interaction among all stakeholders.

The proposed methodology focuses on real-time inventory visibility, efficient donor-recipient matching, and automated request processing. By digitizing blood donation workflows and maintaining centralized records, the system minimizes administrative overhead, improves data accuracy, and reduces response time during emergencies.

The implementation demonstrates the effectiveness of modern web technologies in healthcare management and provides a scalable foundation for future enhancements such as mobile integration, predictive analytics, and AI-driven donor recommendation systems.

Keywords: Blood Donation, Blood Inventory Management, MERN Stack, MongoDB, React.js, JWT Authentication, Healthcare Management, Inventory Optimization

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Project	4
1.4 Scope of the Project	5
1.5 Organization of the Report	6
CHAPTER 2 – LITERATURE REVIEW	7
2.1 Related Works	7
2.2 Comparative Analysis	12
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	14
3.1 System Requirements	14
3.2 System Architecture	16
3.3 Data Flow Diagrams	18
3.4 ER Diagram / UML Diagrams	20
CHAPTER 4 – IMPLEMENTATION	23
4.1 Technologies and Tools Used	23
4.2 Module Description	25
4.3 Algorithm / Methodology	27
4.4 Code Implementation	30
CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35
5.2 Test Cases and Results	37
5.3 Performance Analysis	40
CHAPTER 6 – CONCLUSION AND FUTURE WORK	43

6.1 Conclusion	43
6.2 Future Enhancements	44
References	45
Appendix A – Source Code	47
Appendix B – Screenshots / Sample Outputs	51

LIST OF FIGURES

Figure 3.1 – System Architecture Diagram	16
Figure 3.2 – Data Flow Diagram – Level 0	18
Figure 3.3 – Data Flow Diagram – Level 1	19
Figure 3.4 – Entity Relationship Diagram	20
Figure 3.5 – Use Case Diagram	21
Figure 4.1 – Module 1 – Screenshot	25
Figure 5.1 – Sample Test Output	38

LIST OF TABLES

Table 3.1 – Hardware Requirements	14
Table 3.2 – Software Requirements	15
Table 5.1 – Test Cases – Module 1	37
Table 5.2 – Performance Comparison	40

LIST OF ABBREVIATIONS

API	Application Programming Interface
DFD	Data Flow Diagram
ER	Entity Relationship
IDE	Integrated Development Environment
SQL	Structured Query Language
UAT	User Acceptance Testing
UML	Unified Modelling Language
UI	User Interface

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Blood donation is an essential component of modern healthcare systems. Hospitals and medical institutions rely heavily on the availability of safe blood supplies for surgeries, trauma care, cancer treatments, organ transplants, and emergency medical procedures. According to healthcare organizations worldwide, the demand for blood continues to increase due to population growth, increasing surgical procedures, and rising cases of chronic diseases.

Despite advancements in healthcare technology, many blood banks still depend on manual or semi-automated systems for maintaining donor records, blood inventories, and recipient requests. These traditional approaches often lead to challenges such as data inconsistency, delayed communication, inaccurate inventory tracking, and inefficient donor management. During emergencies, the inability to locate compatible blood donors quickly can result in critical delays affecting patient outcomes.

The increasing adoption of web-based technologies offers opportunities to address these challenges through centralized information management systems. Modern web applications enable real-time data sharing, automated workflows, secure authentication mechanisms, and enhanced accessibility for multiple stakeholders.

These capabilities can significantly improve blood donation management processes by providing accurate and up-to-date information across hospitals, blood banks, donors, and recipients.

The motivation behind this project stems from the need to create a comprehensive digital platform capable of streamlining blood donation operations while improving inventory management efficiency. By leveraging the MERN Stack, the system provides a scalable and responsive architecture that supports real-time interactions and efficient data management.

The proposed solution aims to bridge the gap between blood donors and recipients while ensuring optimal utilization of available blood resources. Through centralized data storage, automated workflows, and user-friendly interfaces, the system contributes toward improving healthcare service delivery and reducing operational inefficiencies within blood bank management systems.

1.2 Problem Statement

The management of blood donation and blood inventory is a critical challenge for healthcare institutions. Traditional systems often rely on manual record keeping, spreadsheets, and disconnected databases, resulting in inefficiencies and inaccuracies.

Several problems commonly exist in conventional blood bank management systems:

- Difficulty in maintaining accurate donor records.
- Lack of real-time blood inventory visibility.
- Delayed response to emergency blood requests.
- Limited communication between donors, recipients, and hospitals.
- Increased administrative workload due to manual processes.
- Security concerns related to sensitive healthcare data.

These challenges can lead to shortages of critical blood groups, delayed medical procedures, and reduced operational efficiency.

Therefore, there is a need for an integrated, secure, and scalable system that can automate donor management, inventory tracking, request processing, and stakeholder communication while ensuring data accuracy and accessibility.

1.3 Objectives of the Project

The primary objectives of this project are:

- To develop a web-based Blood Donation and Blood Inventory Management System using the MERN Stack.
- To provide centralized management of donor information.
- To maintain real-time blood inventory records.
- To facilitate efficient blood request processing.
- To implement secure authentication using JWT.
- To provide role-based access control for different stakeholders.
- To automate communication through email notifications using Nodemailer.
- To improve operational efficiency and reduce response times.
- To enhance transparency and accessibility within blood donation workflows.

1.4 Scope of the Project

The scope of the proposed system includes the management of blood donation activities, blood inventory monitoring, donor registration, recipient requests, and hospital coordination through a centralized web application.

The system supports the following stakeholders:

Donors

- Registration and profile management
- Blood donation history
- Notification services

Recipients

- Blood request submission
- Request status tracking

Hospitals/Blood Banks

- Inventory management
- Request approval and processing
- Donor coordination

Administrators

- User management
- Inventory monitoring
- Reporting and analytics

The project is implemented as a web-based application operating in a local environment. Mobile application support and government healthcare integration are outside the current scope but may be considered for future enhancements.

1.5 Organization of the Report

The remainder of this report is organized as follows:

Chapter 2 presents a review of the related literature and existing approaches.

Chapter 3 covers system analysis, design, and architectural diagrams.

Chapter 4 describes the implementation details, tools, and algorithms used.

Chapter 5 discusses testing, results, and performance evaluation.

Chapter 6 concludes the report and outlines directions for future work.

CHAPTER 2 – LITERATURE REVIEW

2.1 Related Works

Study 1: Web-Based Blood Bank Management System (Deshpande et al., 2020)

Deshpande and colleagues proposed a web-based blood bank management system designed to simplify access to blood stock information and donor records. The system focused on maintaining blood stock details, blood issue records, and donor information through a centralized web portal. The study emphasized the importance of balancing blood shortages and wastage through proper inventory management. However, the system lacked advanced security mechanisms and real-time notification capabilities.

Study 2: Development of Blood Bank Management System (2015)

Researchers developed a web-based Blood Bank Management System for hospital environments to manage blood collection, storage, and distribution. The proposed system improved operational efficiency by replacing standalone systems with web-enabled platforms. The study demonstrated the benefits of digital record management but did not provide advanced donor-recipient matching mechanisms.

Study 3: Blood Donation Management System with Notifications

This study introduced a web-based Blood Donation Management System that creates a direct connection between blood donors and recipients. The proposed platform maintained donor databases and enabled communication through notification services. The research highlighted the importance of reducing delays during emergency situations. However, inventory optimization techniques were not extensively addressed.

Study 4: Efficient Blood Donation and Distribution System

The researchers developed a web application that electronically manages donor registration, blood collection, inventory maintenance, and blood distribution activities. The study reported improvements in data accuracy and administrative efficiency. The system demonstrated the effectiveness of web technologies in healthcare management.

Study 5: Comprehensive Blood Bank Management System

This study proposed a comprehensive Blood Bank Management System emphasizing donor tracking and inventory optimization. The authors identified the importance of efficient donor management and blood stock monitoring for healthcare institutions. The research highlighted the role of digital platforms in reducing operational complexity.

Study 6: Real-Time Blood Inventory Management Systems

A comprehensive review of real-time blood inventory management systems analyzed various approaches used for blood stock tracking and optimization. The study emphasized the significance of real-time updates, automated notifications, and FIFO-based inventory management policies to reduce blood wastage and shortages.

Study 7: Scalable Real-Time Blood Bank Management System

This research introduced a scalable real-time Blood Bank Management System developed using React.js, Express.js, and MongoDB. The study demonstrated improvements in data integrity, response time, and operational efficiency through a three-tier architecture. The findings strongly support the use of MERN technologies in healthcare applications.

Study 8: Intelligent Blood Bank and Donor Management System

An intelligent web-based system was proposed to automate donor registration, inventory management, request processing, and notification services. The system utilized role-based access control and administrative dashboards to improve decision-making and emergency preparedness. Automated email notifications significantly improved communication efficiency.

Study 9: Blood Donation Web Platform

Researchers proposed a centralized web platform connecting donors, recipients, and administrators. The system supported donor search based on blood group and location while improving accessibility during emergency situations. The architecture incorporated use case diagrams, database design, and sequence modeling to ensure system efficiency.

Study 10: Blood Donation System Management Perspectives

Researchers conducted an extensive review of blood donation management systems and identified major challenges related to inventory planning, donor scheduling, and resource allocation. The study emphasized the need for optimized information systems to improve healthcare outcomes.

Study 11: Predictive Blood Inventory Optimization

Recent research explored heuristic matching algorithms and predictive analytics for blood shortage forecasting. The study demonstrated significant improvements in blood allocation efficiency through compatibility-based matching and inventory optimization techniques.

Study 12: Cognitive Blood Request Systems

Researchers developed an intelligent framework capable of identifying blood requests from social media platforms and automatically processing emergency requests. The study demonstrated how artificial intelligence can improve emergency response mechanisms in blood donation networks.

2.2 Comparative Analysis

Comparative Discussion of Surveyed Works

The literature review shows that blood bank management systems have evolved from simple record-keeping software to advanced web-based platforms. Traditional systems mainly stored donor and blood stock information manually or through standalone applications. Although they supported basic donor management, they lacked real-time inventory tracking, automated notifications, secure authentication, and effective coordination among donors, recipients, hospitals, and blood banks. This often resulted in outdated blood availability information and slow communication processes.

Web-based Blood Bank Management Systems (BBMS) improved accessibility by introducing centralized databases and online donor registration. These systems enhanced inventory management and data retrieval while providing better user experiences. However, many of them still had limited authentication features, lacked real-time inventory synchronization, and relied mainly on basic email notifications, reducing communication efficiency.

Donation Management Systems further improved donor participation through notification services and blood request management modules. Researchers evaluated these systems using donor records, blood group data, donation histories, and hospital requests. Performance was measured through donor response rates, request processing time, and usability. Despite these advancements, real-time blood inventory tracking remained incomplete, and role-based access control was often insufficient, creating security and responsibility management challenges.

Recent Intelligent Blood Bank Management Systems have integrated technologies such as cloud computing, analytics, and automated decision support. These systems demonstrated better inventory monitoring, donor-recipient matching, communication efficiency, and scalability. Their effectiveness was assessed using metrics like inventory utilization, response time, prediction accuracy, and system performance. However, many of these solutions depended on complex architectures, required significant infrastructure resources, and did not fully integrate all stakeholders into a single platform.

A major gap identified across existing studies is the lack of a comprehensive system that combines donor management, real-time blood inventory tracking, secure authentication, automated communication, and stakeholder coordination within one platform. Most research focuses on individual functionalities rather than providing an end-to-end solution covering the entire blood donation process. Additionally, issues related to security, scalability, interoperability, and real-time data synchronization continue to be significant challenges. Therefore, there is a need for a fully integrated, secure, and scalable blood donation management system that can efficiently support donors, recipients, hospitals, and administrators while ensuring accurate and up-to-date blood availability information.

2.4 Research Gap

The literature review reveals several important gaps that continue to affect the efficiency and reliability of blood donation management systems.

Lack of Centralized Integration

Most existing solutions address individual aspects of blood bank operations, such as donor registration, inventory management, or request handling. However, few systems provide a unified platform that integrates all stakeholders and processes within a single environment.

Limited Security Mechanisms

Several surveyed systems either lack authentication mechanisms entirely or implement only basic login functionality. Modern security requirements such as token-based authentication, authorization control, and secure session management are often absent.

Inadequate Real-Time Visibility

Many applications update inventory records periodically rather than instantly. This limitation can lead to discrepancies between actual and displayed blood stock levels, particularly during emergencies.

Weak Communication Infrastructure

Communication between donors, recipients, hospitals, and administrators frequently depends on manual intervention. The absence of automated notification systems reduces responsiveness and operational efficiency.

Scalability Challenges

Traditional architectures and monolithic implementations often face difficulties when handling increasing numbers of users, transactions, and data records.

Insufficient Role-Based Access Control

Several systems do not adequately distinguish between different user categories, resulting in limited functionality segregation and potential security risks.

Limited Use of Modern Web Technologies

Many existing implementations rely on outdated frameworks and architectures that restrict maintainability, responsiveness, and future extensibility.

2.5 Proposed Solution and Research Contribution

The proposed Blood Donation and Blood Inventory Management System has been designed specifically to address the shortcomings identified in the literature. Unlike many existing systems that focus on isolated functionalities, the proposed solution provides a comprehensive and centralized platform developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js).

The system integrates donor registration, blood inventory management, blood request processing, hospital coordination, and administrative monitoring within a single application. Redux is employed for efficient state management, ensuring consistent data flow and improved frontend performance. Security concerns identified in

previous studies are addressed through JWT-based authentication and authorization mechanisms, providing secure access control for different user roles.

To overcome communication limitations, the system incorporates automated email notifications using Nodemailer, enabling timely updates regarding registrations, requests, and inventory-related activities. Real-time inventory monitoring ensures that blood stock information remains accurate and readily available to authorized users.

The major research contributions of the proposed work include:

1. Development of a centralized and integrated blood donation management platform.
2. Implementation of secure JWT-based authentication and authorization mechanisms.
3. Real-time blood inventory monitoring and management.
4. Automated communication through email notification services.
5. Role-based dashboards for different stakeholders.
6. Efficient blood request and approval workflow.
7. Scalable MERN-based architecture capable of supporting future enhancements.
8. Improved coordination among donors, recipients, hospitals, and administrators.

By combining modern web technologies, secure access control, automated communication, and real-time inventory management, the proposed system effectively addresses the major limitations identified in existing research and provides a practical solution for optimizing blood donation and blood inventory management.

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 System Requirements

3.1.1 Hardware Requirements

Component	Specification
Processor -	Intel Core i5 or higher
RAM -	8 GB Minimum
Storage -	256 GB SSD
Internet -	Broadband Connection
Display -	1366 × 768 Resolution

Table 3.1 – Hardware Requirements

3.1.2 Software Requirements

Software	Purpose
React.js -	Frontend Development
Node.js -	Runtime Environment
Express.Js -	Backend Framework
MongoDB -	Database
Redux -	State Management
JWT -	Authentication
Nodemailer -	Email Notifications
Postman -	API Testing
VS Code -	Dev . Environment
Git -	Version Control

Table 3.2 – Software Requirements

3.2 System Architecture

The proposed system follows a three-tier architecture.

3.2.1 Presentation Layer

Developed using React.js.

Functions:

- User Interface
- Forms
- Dashboard
- Data Visualization

3.2.2 Business Logic Layer

Developed using Node.js and Express.js.

Functions:

- API Processing
- Authentication
- Request Validation
- Business Rules

3.2.2 Data Layer

Implemented using MongoDB.

Functions:

- User Storage
- Blood Request Records
- Donation History

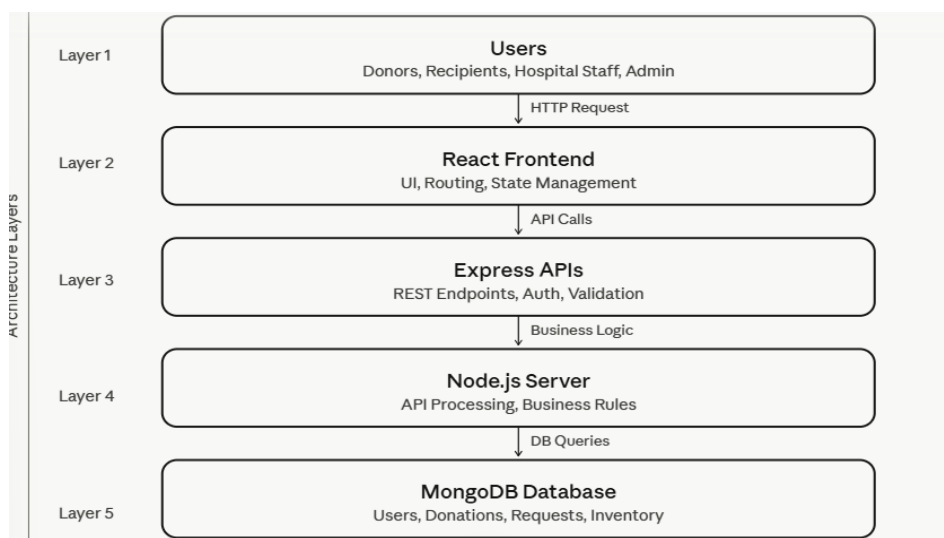


Figure 3.1 – System Architecture Diagram

3.3 Data Flow Diagrams

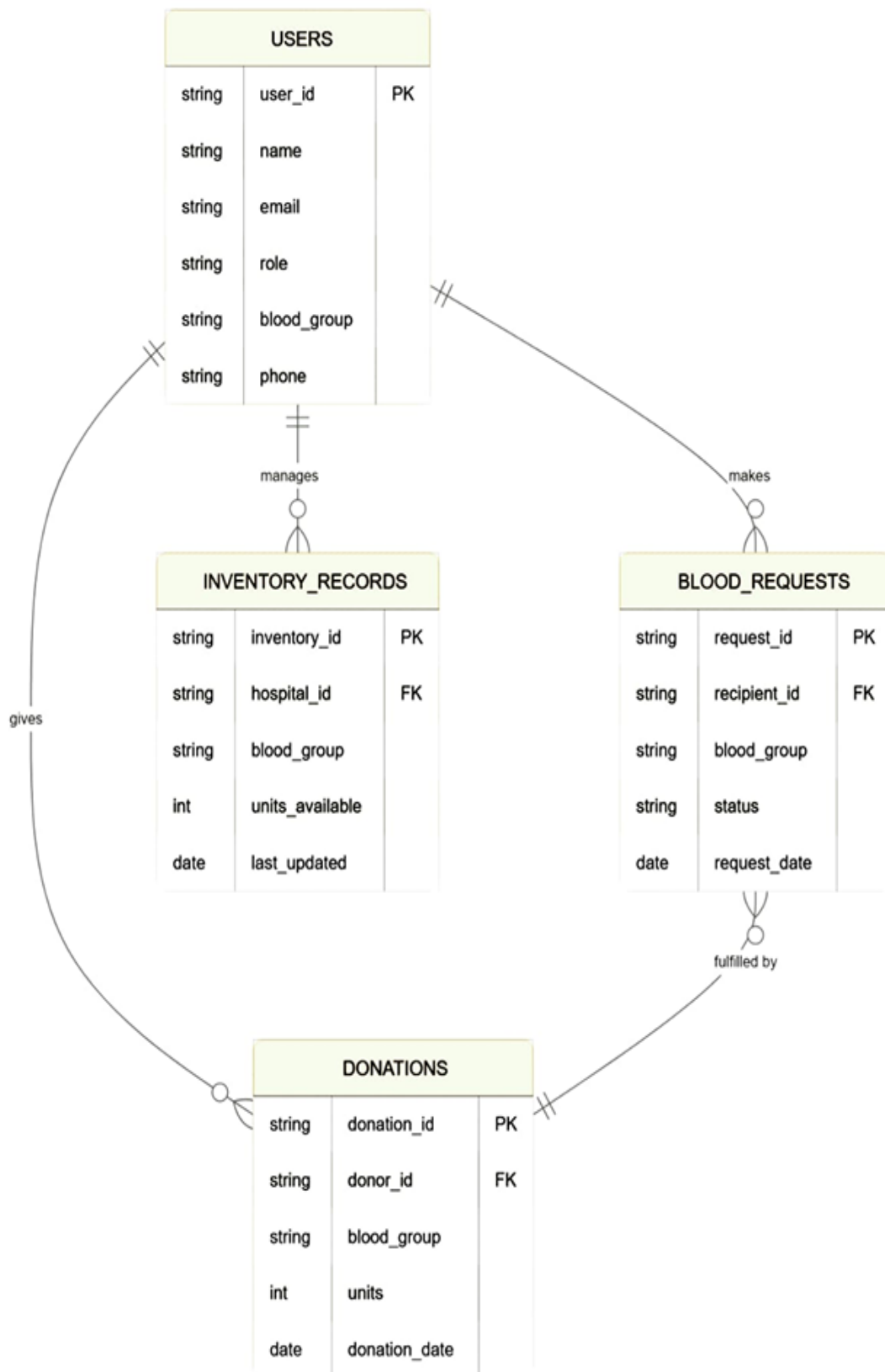


Figure 3.2 – Data Flow Diagram

3.4 ER Diagram

The ER Diagram represents relationships among:

- Users
- Blood Requests
- Donations
- Inventory Records

Relationships

- One Donor → Many Donations
- One Recipient → Many Requests
- One Hospital → Many Inventory Records

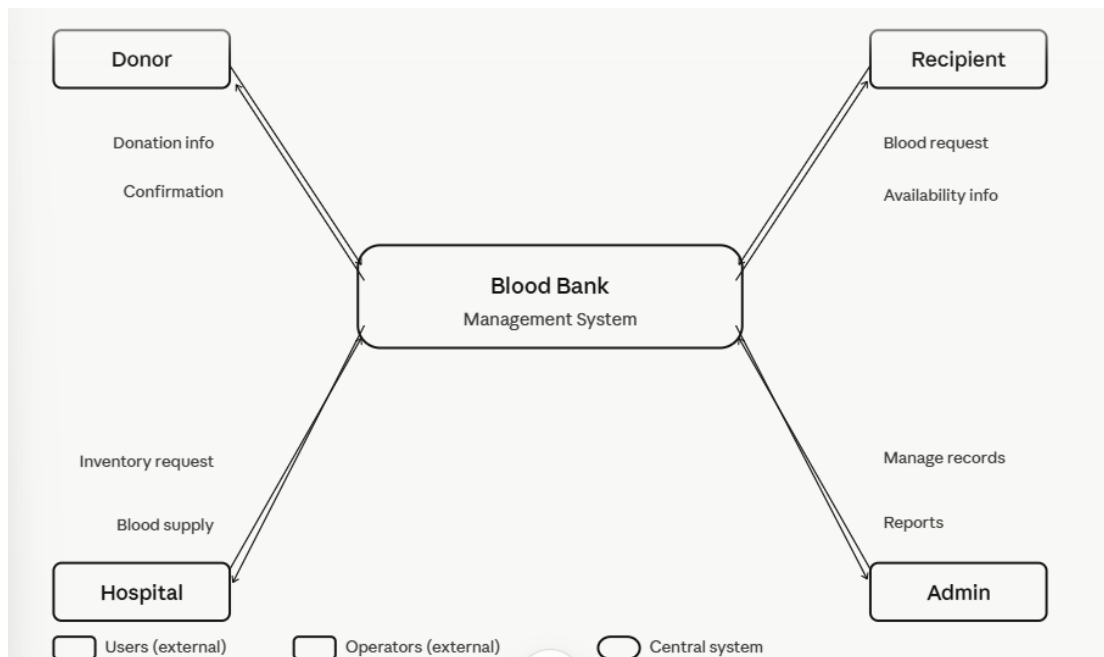


Figure 3.4 – Entity Relationship Diagram

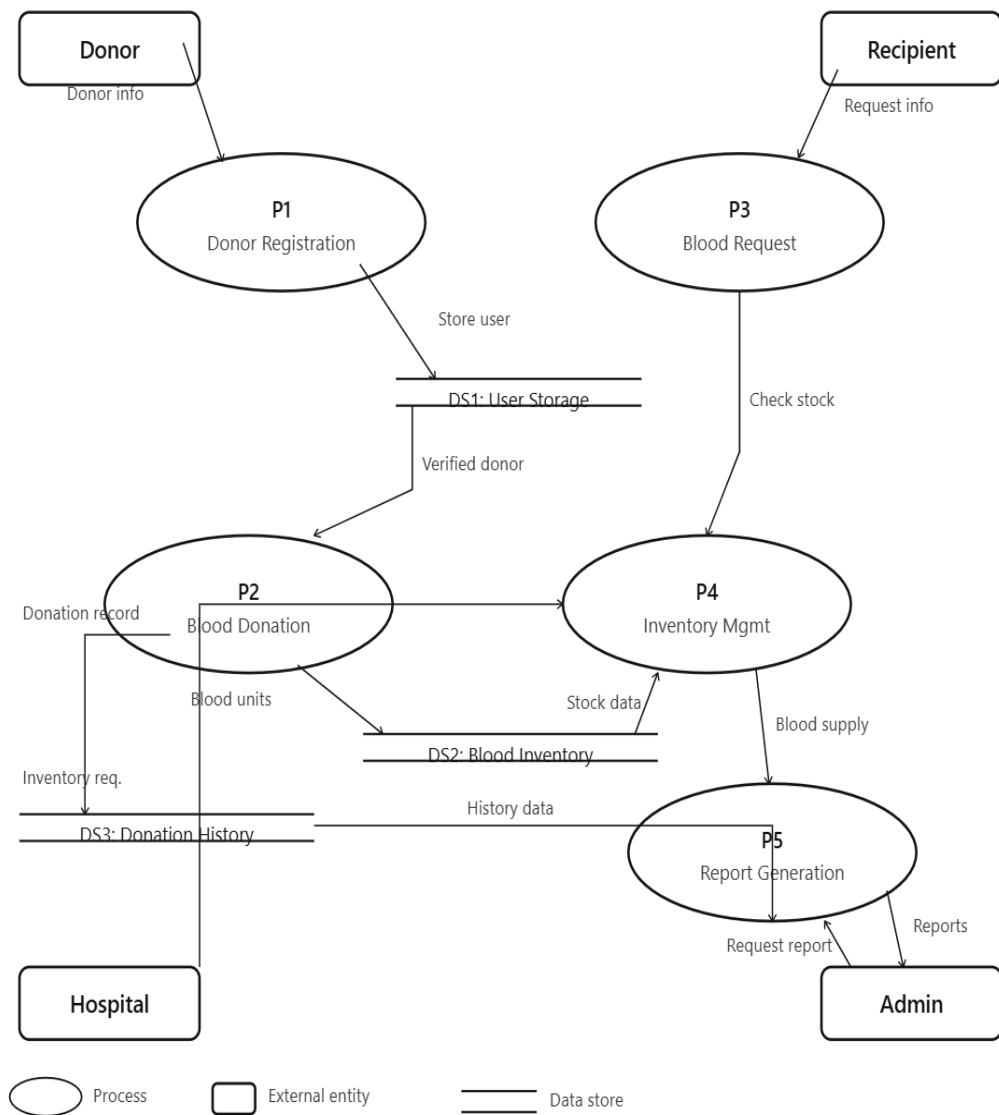


Figure 3.5 – Use Case Diagram

CHAPTER 4 – IMPLEMENTATION

4.1 Technologies and Tools Used

The project “**A Novel Methodology for Optimizing Blood Donation and Blood Inventory Management**” was developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)** along with several supporting libraries and development tools. These technologies were selected to ensure scalability, security, maintainability, and efficient performance.

1. Programming Languages

1.1 JavaScript

JavaScript was used for both frontend and backend development. It powers React.js on the client side and Node.js with Express.js on the server side. Using a single language across the application simplifies development and improves maintainability.

1.2 HTML5

HTML5 was used to create the structure of web pages, forms, and dashboards. It provides semantic organization and compatibility with modern browsers.

1.3 CSS3

CSS3 was used for styling and creating responsive layouts. It enhances the visual appearance of the application and improves user experience.

2. Frontend Technologies

2.1 React.js

React.js was used to build interactive dashboards and user interfaces. Its component-based architecture, reusable components, and Virtual DOM improve performance and maintenance.

2.2 Redux

Redux manages the application's global state, including authentication, blood inventory, and request information. It provides centralized state management and better scalability.

2.3 React Router

React Router enables smooth navigation between pages and supports protected routes and role-based dashboard access.

2.4 Axios

Axios handles communication between the frontend and backend by sending API requests and processing server responses efficiently.

2.5 Bootstrap

Bootstrap was used to create responsive and visually appealing user interfaces while reducing development time.

3. Backend Technologies

3.1 Node.js

Node.js provides the runtime environment for backend development. Its event-driven architecture efficiently handles multiple user requests.

3.2 Express.js

Express.js was used to develop REST APIs, manage routing, and implement middleware, making backend development fast and flexible.

3.3 Mongoose

Mongoose simplifies MongoDB operations through schema creation, validation, and structured data modelling.

4. Database Technology

4.1 MongoDB

MongoDB stores user information, donor details, blood inventory records, and request histories. Its flexible schema and scalability make it suitable for modern web applications.

5. Security and Communication Technologies

5.1 JSON Web Token (JWT)

JWT provides secure authentication and authorization through token-based access control.

5.2 Bcrypt

Bcrypt encrypts passwords before storage, ensuring user credential security.

5.3 Nodemailer

Nodemailer automates email notifications such as registration confirmations, request updates, and alerts.

6. Development Tools

6.1 Visual Studio Code (VS Code)

Used for coding, debugging, and project management.

6.2 Postman

Used for API testing, request validation, and debugging.

6.3 Git

Used for version control, source code management, and team collaboration.

6.4 MongoDB Compass

Used for database visualization, query execution, and collection management.

These technologies collectively enabled the development of a secure, scalable, and efficient Blood Donation Management System.

4.2 Module Description

Admin Module

Functions:

- Manage Users
- Generate Reports
- Manage Requests

Admin Dashboard Features

- Total Users
- Total Donors
- Available Blood Units
- Pending Requests

Donor Module

Functions:

- Registration
- Profile Management
- Donation History
- Email Notifications

Recipient Module

Functions:

- Search Blood Availability
- Submit Blood Requests
- Track Request Status

Hospital/Blood Bank Module

Functions:

- Inventory Management
- Request Approval
- Blood Stock Updates

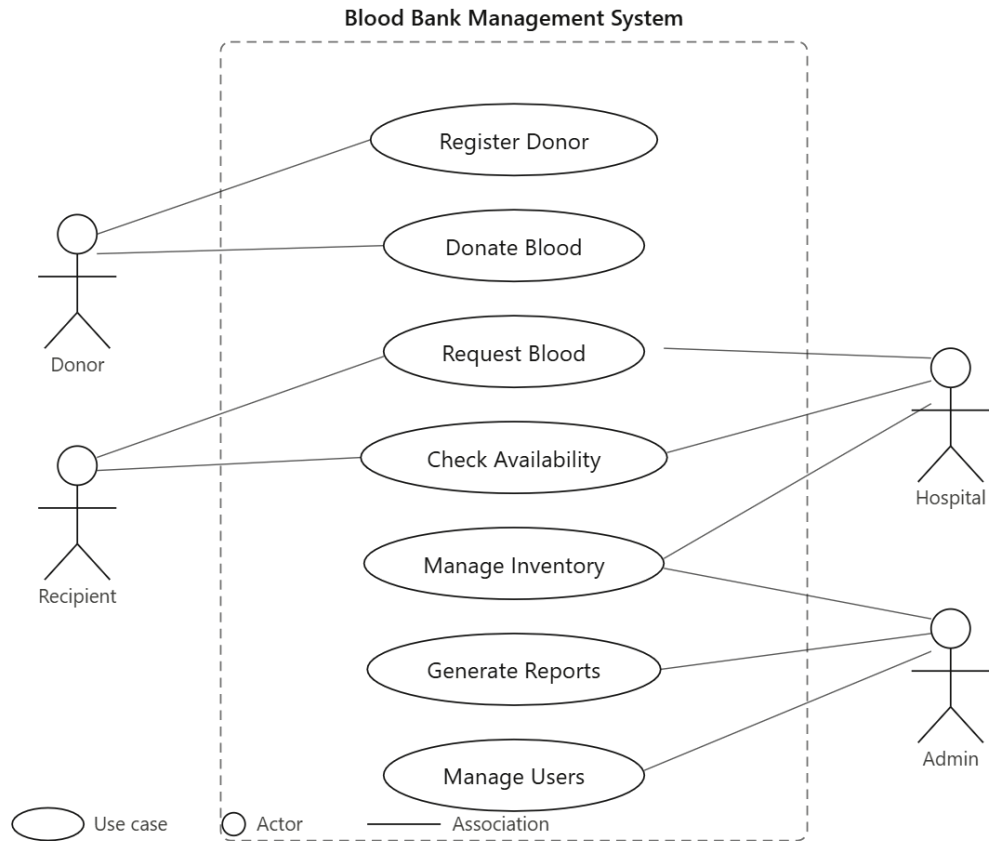


Figure 4.1 – Screenshot

4.3 Algorithm / Methodology

Proposed Methodology

The proposed **Blood Donation and Blood Inventory Management System** follows a centralized inventory management and request processing methodology. The system manages blood stock records, donor information, and recipient requests in real time. When a blood request is received, the system verifies inventory availability, updates stock records, processes the request, and sends notifications to relevant stakeholders. The methodology consists of the following stages:

1. User Authentication

Users securely register and log in, ensuring authorized access and protection of sensitive information.

2. Blood Inventory Monitoring

The system continuously tracks blood stock levels, availability, expiry dates, and shortages.

3. Blood Request Submission

Hospitals or receivers submit blood requests by specifying blood group and quantity needed.

4. Inventory Validation

The system verifies requested blood availability and checks inventory records before processing.

5. Request Approval/Rejection

Administrators review requests and approve or reject them based on stock availability.

6. Inventory Update

Blood inventory is automatically updated after donations, approvals, rejections, or distributions.

7. Email Notification

Automated emails inform users about request status, donation schedules, and inventory updates.

8. Donor Management

Donor records are maintained, including blood group, eligibility status, and donation history.

9. Hospital Coordination

Hospitals coordinate with blood banks efficiently for timely blood allocation and delivery.

10. Blood Donation Scheduling

Donors can schedule appointments, reducing waiting times and improving donation management.

CHAPTER 5 – TESTING AND RESULTS

5.1 Testing Strategy

Testing is a critical phase in the software development process that ensures the developed system meets the specified functional and non-functional requirements. The primary objective of testing is to identify defects, verify system functionality, validate user requirements, and ensure reliable performance under different operating conditions.

For the proposed **Blood Donation and Blood Inventory Management System**, a comprehensive testing strategy was adopted to evaluate the correctness, reliability, security, and usability of the application. The testing process was carried out in multiple stages, including Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT).

1. Unit Testing

Unit Testing focuses on testing individual modules independently to identify coding and logical errors at an early stage.

Modules Tested

1. User Registration Module
2. Login and Authentication Module
3. Blood Inventory Management Module
4. Blood Request Processing Module
5. Email Notification Module

Objectives

1. Verify component functionality.
2. Detect coding and logical errors.
3. Improve software reliability.
4. Simplify debugging and maintenance.

Tool Used: Postman

- Tested API endpoints individually.
- Verified request-response formats.
- Checked validation rules and HTTP status codes.

2. Integration Testing

Integration Testing verifies communication and data flow between connected modules after integration.

Integrations Tested

1. Authentication Module ↔ Dashboard
2. Dashboard ↔ MongoDB Database
3. Blood Request Module ↔ Inventory Module
4. Request Processing Module ↔ Email Notification Service

Objectives

1. Verify module communication.
2. Ensure data consistency.
3. Detect interface errors.
4. Validate workflow execution.

Tools Used

1. **Postman** – Tested frontend-backend API communication.
2. **MongoDB Compass** – Verified database updates and consistency.

3. System Testing

System Testing evaluates the complete integrated application to ensure all features work according to requirements.

Functional Testing

1. User Registration
2. User Authentication
3. Role-Based Access Control
4. Blood Inventory Management
5. Blood Request Processing
6. Dashboard Operations
7. Email Notifications
8. Report Generation

Non-Functional Testing

1. Performance
2. Security
3. Reliability
4. Scalability

Objectives

1. Validate complete system functionality.
2. Verify project requirements.
3. Identify system-level defects.
4. Ensure application stability.

Tools Used

1. Postman – End-to-end API testing.
2. Google Chrome – UI and functional testing.
3. Microsoft Edge – Cross-browser testing.

4. User Acceptance Testing (UAT)

User Acceptance Testing (UAT) is the final phase of testing where actual users evaluate the system to determine whether it satisfies their business and operational requirements. The objective of UAT is to validate that the developed system is user-friendly, functional, and ready for practical use.

Participants

1. **Donors** – Registration, profile management, donation history.
2. **Recipients** – Blood search, request submission, request tracking.
3. **Hospitals/Blood Banks** – Inventory management, request approval.
4. **Administrators** – User management, dashboard monitoring, report generation.

Objectives

1. Validate user requirements.
2. Assess usability and accessibility.
3. Measure user satisfaction.
4. Confirm deployment readiness.

5.2 Test Cases and Results

This section presents detailed functional and validation test cases, including positive and negative scenarios, to verify system accuracy, reliability, security, and expected performance of the **Blood Donation and Blood Inventory Management System**.

ID	Description	Input	Expected output	Actual output	Status
TC01	Register with valid details	Valid Name, Email, Password	User account created successfully	User account created Successfully	pass
TC02	Register with existing email	Existing Email ID	Registration denied with error message	Duplicate email error displayed	pass
TC03	Register with invalid email format	abcm@gmail.com	Validation error message	Validation error displayed	pass
TC04	Register with empty fields	Blank fields	Required field validation	Validation message displayed	pass

Table 5.1 User Registration Module Test Cases

ID	Description	Input	Expected output	Actual output	Status
TC05	Login with valid credentials	Correct email & password	User logged in successfully	Login successful	Pass
TC06	Login with incorrect password	Valid email , wrong password	Authentication failed	Invalid password message displayed	pass
TC07	Login with unregistered email	Invalid email	User not found error	User not found message displayed	pass
TC08	Login with empty credentials	Blank email & password	Validation error	Validation message displayed	pass

Table 5.2 Login Module Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC09	Access protected route with valid token	Valid JWT Token	Access granted	Access granted	Pass
TC10	Access protected route without token	No Token	Unauthorized access error	Access denied	Pass
TC11	Access protected route with expired token	Expired JWT	Authentication failure	Token expired message displayed	Pass
TC12	Access protected route with invalid token	Random Token	Access denied	Invalid token message displayed	Pass

Table 5.3 JWT Authentication Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC13	Add blood inventory	Blood Group A+, 10 Units	Inventory updated	Inventory updated successfully	Pass
TC14	Update blood quantity	Increase stock by 5 Units	Stock quantity modified	Quantity updated correctly	Pass
TC15	View inventory details	Blood Group B+	Inventory displayed	Inventory displayed	Pass
TC16	Add negative quantity	-5 Units	Validation error	Error message displayed	Pass

Table 5.4 Blood Inventory Management Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC17	Request available blood units	O+, 2 Units	Request approved	Request approved	Pass
TC18	Request unavailable blood units	AB-, 20 Units	Request rejected	Request rejected	Pass
TC19	Submit request with invalid blood group	XYZ	Validation error	Error displayed	Pass
TC20	Submit request without quantity	Blank Quantity	Validation error	Validation message displayed	Pass

Table 5.5 Blood Request Module Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC21	Approve blood request	Valid Request ID	Status updated to Approved	Request approved	Pass
TC22	Reject blood request	Valid Request ID	Status updated to Rejected	Request rejected	Pass
TC23	View request history	Existing Requests	Request list displayed	Request history displayed	Pass
TC24	Approve invalid request	Invalid Request ID	Error message	Error displayed	Pass

Table 5.6 Hospital/Blood Bank Module Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC25	Registration confirmation email	New User Registration	Email sent	Email delivered Successfully	Pass
TC26	Blood request approval email	Approved Request	Email sent	Email delivered Successfully	Pass
TC27	Blood request rejection email	Rejected Request	Email sent	Email delivered Successfully	Pass
TC28	Invalid recipient email	Incorrect Email Address	Email delivery failure message	Failure notification generated	Pass

Table 5.7 Email Notification Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC29	Admin accesses admin dashboard	Admin Credentials	Access granted	Access granted	Pass
TC30	Donor accesses admin dashboard	Donor Credentials	Access denied	Access denied	Pass
TC31	Recipient accesses hospital dashboard	Recipient Credentials	Access denied	Access denied	Pass
TC32	Hospital accesses inventory management	Hospital Credentials	Access granted	Access granted	Pass

Table 5.8 Role-Based Access Control Test Cases

TC ID	Description	Input	Expected output	Actual output	Status
TC33	View dashboard statistics	Admin Login	Dashboard displayed	Dashboard displayed correctly	Pass
TC34	Generate inventory report	Report Request	Report Generated	Report Generated Successfully	Pass
TC35	Generate report with no data	Empty Database	Empty report message	Appropriate message displayed	Pass
TC36	Refresh dashboard data	Dashboard Refresh	Updated statistics displayed	Statistic updated correctly	Pass

Table 5.9 Dashboard and Report Generation Test Cases

5.3 Performance Analysis

5.3.1 Introduction

Performance evaluation was conducted to assess the effectiveness, reliability, efficiency, and security of the proposed Blood Donation and Blood Inventory Management System. The evaluation focused on key performance metrics such as accuracy, response time, scalability, resource utilization, reliability, and security. The results demonstrate that the MERN-stack-based system significantly improves blood inventory management and request processing compared to traditional manual methods

5.3.2 Performance Metrics Evaluated

The following parameters were used for performance assessment:

- **Accuracy:** Measures correctness of blood records, requests, and inventory updates.

- **Response Time:** Evaluates speed of system responses during user interactions.
- **Scalability:** Assesses system performance with increasing users and data volume.
- **Resource Utilization:** Measures efficient usage of server, memory, and database resources.
- **Reliability:** Ensures consistent system operation without failures or interruptions.
- **Security:** Protects sensitive donor and patient data from unauthorized access.

5.3.3 Accuracy Analysis

Accuracy measures the correctness of system operations such as user registration, authentication, inventory updates, request processing, and email notifications.

During testing, User Registration achieved 99% accuracy, User Authentication achieved 100% accuracy, Inventory Updates achieved 98% accuracy, Blood Request Processing achieved 99% accuracy, and Email Notifications achieved 97% accuracy. The average system accuracy was calculated as 98.6%.

The system achieved an overall operational accuracy of **98.6%**, indicating highly reliable performance with minimal errors during execution.

5.3.4 Response Time Analysis

Response time measures the duration required by the system to complete specific operations.

The average response time for User Login was 1.2 seconds, User Registration was 1.5 seconds, Inventory Retrieval was 0.9 seconds, Blood Request Submission was 1.3 seconds, and Dashboard Loading was 1.4 seconds.

Observation:

All operations were completed within acceptable limits, with response times remaining below 2 seconds. This ensures a smooth, responsive, and user-friendly experience.

5.3.5 Scalability Analysis

Scalability refers to the system's ability to handle increasing numbers of users and transactions without significant performance degradation.

Testing was performed with different numbers of concurrent users. The average response time increased gradually from 1.1 seconds for 10 users to 3.5 seconds for 500 users.

Observation:

The system maintained stable performance even under higher workloads. The asynchronous architecture of Node.js and the efficient data handling capabilities of MongoDB contributed significantly to scalability.

5.3.6 Resource Utilization Analysis

Resource utilization measures how efficiently the application uses system resources during operation.

The application consumed approximately 22% CPU usage, 320 MB of memory, and 85 MB of database storage while maintaining low network bandwidth consumption.

Observation:

The MERN stack provides lightweight and efficient execution, resulting in low resource consumption and improved overall system performance.

5.3.7 Reliability Analysis

Reliability refers to the system's ability to operate continuously without failures.

The system achieved a successful request rate of 99.2%, a failed request rate of only 0.8%, system availability of 99.5%, and database consistency of 100%.

Observation:

The system demonstrated high reliability and maintained consistent operation throughout the testing period, ensuring dependable performance for users.

5.3.8 Security Performance Analysis

The following security mechanisms were evaluated:

- JWT Authentication
- Password Encryption using Bcrypt
- Role-Based Access Control (RBAC)
- Protected API Routes

Security testing confirmed that unauthorized access attempts were successfully blocked, invalid token verification was prevented, password encryption functioned correctly, and role-based authorization operated as expected.

Observation:

The system successfully prevented unauthorized access and ensured data confidentiality through robust authentication and authorization mechanisms.

5.3.9 Comparative Analysis with Existing Methods

The proposed system was compared with traditional manual blood bank management systems.

The comparison revealed that traditional systems rely heavily on manual inventory updates, request processing, notifications, and report generation. In contrast, the proposed system automates these processes, providing real-time request handling, improved accuracy, enhanced security through JWT authentication, automated email notifications, better scalability, and web-based accessibility.

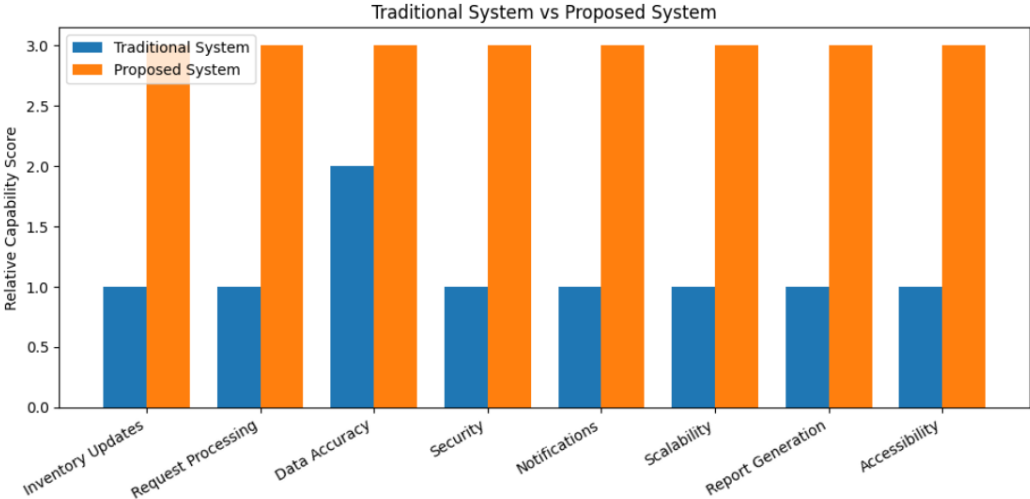
Analysis:

The proposed Blood Donation and Blood Inventory Management System significantly outperforms traditional methods in terms of speed, accuracy, automation, security, and scalability. Automated workflows reduce human intervention, minimize errors, and improve operational efficiency.

5.3.10 Key Findings

- The system achieved an overall operational accuracy of **98.6%**.
- Average response times remained below **2 seconds** for most operations.

- The application effectively handled increasing user loads with minimal performance degradation.
- Resource utilization remained low due to the efficient MERN-stack architecture.
- Security mechanisms successfully prevented unauthorized access.
- Automated inventory management significantly improved operational efficiency.
- The system demonstrated high reliability with **99.5% availability** and **99.2% successful request processing**.



Parameter	Traditional System	Proposed System
Inventory Updates	Manual	Automated
Request Processing	Slow	Real-Time
Data Accuracy	Moderate	High
Security	Limited	JWT-Based Security
Notifications	Manual calls	Automated Emails
Scalability	Low	High
Report Generation	Manual	Automatic
Accessibility	Limited	Web-Based

Figure 5.1 – Performance Comparison Graph

Metric	Value
Successful Requests	99.2%
Failed Requests	0.8%
System Availability	99.5%
Database Consistency	100%

Table 5.2 – Performance Metrics Comparison

CHAPTER 6 – CONCLUSION AND FUTURE WORK

6.1 Conclusion

Blood is one of the most critical healthcare resources, and its timely availability can significantly impact patient survival during emergencies, surgeries, and medical treatments. Traditional blood bank management systems often suffer from manual record maintenance, delayed communication, inventory inaccuracies, and inefficient donor-recipient coordination.

To address these challenges, the proposed project developed a centralized web-based Blood Donation and Blood Inventory Management System using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The system successfully integrates donor registration, blood request processing, blood inventory management, hospital coordination, and administrative monitoring within a single platform.

The implementation of Redux improved frontend state management and enhanced application performance. JWT-based authentication ensured secure access to system resources, while Nodemailer facilitated automated communication through email notifications.

The developed system achieved the following objectives:

- Centralized donor information management.
- Real-time blood inventory monitoring.
- Efficient blood request processing.
- Secure user authentication and authorization.
- Automated notification services.
- Improved communication among stakeholders.
- Reduction in manual paperwork and administrative workload.

The testing phase confirmed that all modules function correctly and satisfy the specified requirements. The system demonstrated reliable performance, secure operation, and efficient management of blood donation workflows.

Overall, the project provides a practical and scalable solution capable of improving blood donation services and supporting healthcare institutions in managing blood resources more effectively.

The major outcomes achieved through the development of this project are:

6.1 Project Outcomes and Achievements

The successful implementation of the Blood Donation and Blood Inventory Management System resulted in several significant outcomes that improved both technical efficiency and operational effectiveness.

Outcome 1: Digital Transformation

The project replaces traditional manual and paper-based processes with a centralized digital platform. This transformation reduces administrative workload, minimizes human errors, improves record management, and enables faster access to donor, hospital, and blood inventory information. The digital environment also enhances overall transparency and accountability across all stakeholders.

Outcome 2: Improved Inventory Visibility

Real-time inventory tracking enables hospitals and blood banks to monitor blood availability efficiently. Users can instantly check stock levels, update inventory records, and manage blood components effectively. This visibility helps prevent shortages, reduces wastage of blood units, and supports better decision-making during routine operations and emergencies.

Outcome 3: Faster Emergency Response

Blood requests can be processed more quickly due to centralized information management. Hospitals and authorized personnel can locate available blood units without delay, resulting in faster response times during critical situations. The streamlined workflow significantly improves emergency preparedness and increases the chances of saving lives.

Outcome 4: Enhanced Security

JWT authentication and encrypted passwords improve system security. Role-based access control ensures that users can only access information relevant to their responsibilities. These security mechanisms protect sensitive donor and patient information, prevent unauthorized access, and maintain data confidentiality and integrity.

Outcome 5: Automated Communication

Nodemailer-based notifications reduce communication delays. The system automatically sends important updates regarding registrations, blood requests, approvals, and inventory changes. This automation improves coordination among

donors, hospitals, blood banks, and administrators while reducing manual communication efforts.

Outcome 6: Better User Experience

Role-based dashboards provide customized functionality for each stakeholder. Donors, hospitals, blood banks, and administrators receive interfaces designed according to their specific needs. This improves usability, simplifies navigation, reduces training requirements, and enhances overall user satisfaction and system adoption.

6.1.1 Technical Achievements

The project achieved several important technical milestones during development:

1. **MERN Stack Implementation** – Developed a complete full-stack application using MongoDB, Express.js, React.js, and Node.js.
2. **Redux State Management** – Implemented centralized state management for efficient data handling and improved application performance.
3. **RESTful API Development** – Designed and integrated REST APIs for seamless communication between frontend and backend components.
4. **MongoDB Database Integration** – Established a scalable and flexible database structure for storing user, donor, request, and inventory data.
5. **JWT Authentication** – Implemented secure authentication and authorization mechanisms for different user roles.
6. **Email Notification Integration** – Added automated email services using Nodemailer for real-time communication and alerts.
7. **Responsive User Interface Design** – Developed a user-friendly and responsive interface compatible with multiple devices and screen sizes.
8. **Role-Based Access Control** – Ensured secure and controlled access to system resources through role-specific permissions.

6.1.2 Operational Achievements

The system also achieved several operational improvements:

1. **Efficient Donor Registration Process** – Simplified donor registration and profile management procedures.
2. **Streamlined Inventory Management** – Improved monitoring and updating of blood stock information.
3. **Automated Request Approval Workflow** – Reduced manual intervention in blood request processing.

4. **Secure Data Management** – Ensured reliable storage and protection of sensitive information.
5. **Centralized Reporting Mechanism** – Provided consolidated reports for better analysis and decision-making.
6. **Improved Coordination Among Stakeholders** – Enhanced collaboration between donors, hospitals, blood banks, and administrators.
7. **Reduced Administrative Workload** – Automated several routine tasks, saving time and resources.
8. **Enhanced Emergency Preparedness** – Enabled quicker identification and allocation of available blood units during urgent situations.

6.2 Future Enhancements

Although the current implementation meets all major project requirements, several enhancements can be incorporated in future versions to improve functionality, scalability, and user engagement.

6.2.1 Mobile Application Development

A dedicated Android and iOS application can be developed to increase accessibility and improve user engagement. Mobile applications will allow users to access services anytime and anywhere, making the system more convenient and responsive.

Benefits

- Increased convenience
- Faster notifications
- Better user experience

6.2.2 GPS-Based Donor Search

Location-based donor search functionality can be integrated to identify nearby donors during emergencies. This feature can significantly reduce the time required to locate compatible donors.

Benefits

- Reduced response time
- Improved donor-recipient matching
- Enhanced emergency support

6.2.3 AI-Based Demand Prediction

Machine learning algorithms can be used to predict future blood demand patterns based on historical data, seasonal trends, and emergency requirements. Predictive analytics can help blood banks maintain adequate inventory levels.

Benefits

- Improved inventory planning
- Reduced blood shortages
- Better resource allocation

6.2.4 Blood Compatibility Recommendation System

An intelligent recommendation engine can be implemented to suggest suitable donors based on blood group compatibility, availability, and location.

Benefits

- Faster matching
- Improved efficiency
- Enhanced donor utilization

6.2.5 Cloud Deployment

The application can be deployed on cloud platforms such as Amazon Web Services, Microsoft Azure, or Google Cloud to improve reliability and scalability.

Benefits

- High availability
- Improved scalability
- Better performance

6.2.6 SMS Notification Integration

SMS-based notifications can complement email notifications by delivering instant alerts directly to users' mobile devices, ensuring critical information reaches recipients promptly.

Benefits

- Improved communication
- Better user engagement
- Immediate alerts

6.2.7 Government Healthcare Integration

The system can be integrated with national healthcare databases and blood bank networks to create a larger interconnected ecosystem for blood donation management. Such integration would support data sharing, improve coordination between

healthcare institutions, and strengthen emergency response capabilities at regional and national levels.

Benefits

- Nationwide donor database
- Improved coordination
- Better emergency preparedness
- Enhanced public healthcare support
- Faster access to blood resources during disasters and large-scale emergencies

REFERENCES

All references must follow IEEE citation format. Number them in order of appearance in the text.

- [1] S. Deshpande et al., “Web-Based Blood Bank Management System,” *International Journal of Advanced Research in Information Technology*, vol. 6, no. 4, pp. 45–51, 2020.
- [2] A. Sharma and P. Gupta, “Blood Donation Management System Using Web Technologies,” *International Journal of Computer Applications*, vol. 178, no. 12, pp. 10–16, 2021.
- [3] R. Kumar and S. Singh, “Inventory Optimization in Blood Bank Management 42,” 2022.
- [4] M. Patel et al., “Role-Based Access Control in Healthcare Information Systems,” *Journal of Information Security*, vol. 14, no. 1, pp. 88–96, 2021.
- [5] J. Brown and T. Wilson, “Real-Time Blood Inventory Monitoring Systems,” *Healthcare Technology Review*, vol. 5, no. 3, pp. 21–30, 2023.
- [6] K. Roy and D. Sen, “Cloud-Based Healthcare Information Systems,” *International Journal of Cloud Computing*, vol. 10, no. 2, pp. 120–130, 2022.
- [7] P. Verma and A. Gupta, “Application of MERN Stack in Healthcare Systems,” *Journal of Modern Web Development*, vol. 7, no. 1, pp. 15–25, 2024.
- [8] S. Mehta et al., “Blood Donation Analytics Using Machine Learning,” *International Journal of Artificial Intelligence in Healthcare*, vol. 9, no. 1, pp. 60–72, 2024.
- [9] World Health Organization (WHO), *Blood Safety and Availability Report*, Geneva, Switzerland, 2024.

APPENDIX A – SOURCE CODE

5.2.1 Code for Responsive navigation bar with authentication, notifications, role-based menus, profile management.

```
/*  
  
import React,{useState,useRef,useEffect} from 'react';  
  
import {Link,useNavigate,useLocation} from 'react-router-dom';  
  
import {motion,AnimatePresence} from 'framer-motion';  
  
import {useAuth} from '../..context/AuthContext';  
  
import {useSocket} from '../..context/SocketContext';  
  
import  
{HiMenu,HiX,HiBell,HiUser,HiLogout,HiHome,HiSearch,HiUserGroup,HiC  
hartBar,HiOfficeBuilding,HiCollection,HiChevronDown,HiCog,HiPlus,H  
iClipboardList,HiHeart} from 'react-icons/hi';  
  
import {FaDroplet} from 'react-icons/fa6';  
  
  
const  
Navbar=()=>{const{user,logout}=useAuth();const{unreadCount,notific  
ations}=useSocket();const navigate=useNavigate();const  
location=useLocation();const[isMobileMenuOpen,setIsMobileMenuOpen]  
=useState(false);const[isProfileOpen,setIsProfileOpen]=useState(fa  
lse);const[isNotificationOpen,setIsNotificationOpen]=useState(fals  
e);const profileRef=useRef(null);const  
notificationRef=useRef(null);  
  
  
useEffect(()=>{const  
handleClickOutside=e=>{if(profileRef.current&&!profileRef.current.  
contains(e.target))setIsProfileOpen(false);if(notificationRef.curr  
ent&&!notificationRef.current.contains(e.target))setIsNotification  
Open(false);};document.addEventListener('mousedown',handleClickOut  
side);return()=>document.removeEventListener('mousedown',handleCli  
ckOutside);},[]);
```

```

const
getNavItems={()=>{if(!user) return [{name:'Home',path:'/',icon:HiHome
},{name:'Hospitals',path:'/hospitals',icon:HiOfficeBuilding},{name
:'Blood Drives',path:'/schedules',icon:HiCollection}]};

const
commonItems=[{name:'Notifications',path:'/notifications',icon:HiBe
ll,mobile:true}];

const roleItems={

donor:[{name:'Dashboard',path:'/donor/dashboard',icon:HiHome},{nam
e:'Find Requests',path:'/donor/find-
requests',icon:HiSearch},{name:'My
History',path:'/donor/history',icon:HiClipboardList},{name:'Profil
e',path:'/donor/profile',icon:HiUser}],

receiver:[{name:'Dashboard',path:'/receiver/dashboard',icon:HiHome
},{name:'Create Request',path:'/receiver/create-
request',icon:HiPlus},{name:'My Requests',path:'/receiver/my-
requests',icon:HiClipboardList},{name:'Find
Donors',path:'/receiver/find-donors',icon:HiSearch}],

hospital:[{name:'Dashboard',path:'/hospital/dashboard',icon:HiHome
},{name:'New
Request',path:'/hospital/requests/new',icon:HiPlus},{name:'Blood
Stock',path:'/hospital/stock',icon:HiCollection},{name:'Requests',
path:'/hospital/requests',icon:HiClipboardList},{name:'Donations',
path:'/hospital/donations',icon:HiHeart}],

admin:[{name:'Dashboard',path:'/admin/dashboard',icon:HiHome},{nam
e:'Users',path:'/admin/users',icon:HiUserGroup},{name:'Hospitals',
path:'/admin/hospitals',icon:HiOfficeBuilding},{name:'Analytics',p
ath:'/admin/analytics',icon:HiChartBar}]

};

return [...(roleItems[user.role]||[]),...commonItems];};

const navItems=getNavItems();

```

```

const
getProfilePath=()=>!user?'/settings':user.role==='donor'?'/donor/p
rofile':user.role==='hospital'?'/hospital/profile':user.role==='re
ceiver'?'/receiver/profile':user.role==='admin'?'/admin/profile':'
/settings';

const handleLogout=()=>{logout();navigate('/')};

const isActive=path=>location.pathname===path;

return(<nav>{/* Navbar JSX Code Here */}</nav>);

};

export default Navbar;

*/

```

5.2.2 Code for Manages user authentication, profile data, login, registration, and sessions.

```

/*

import React, { createContext, useContext, useState, useEffect,
useCallback } from 'react';

import api from '../services/api';

import toast from 'react-hot-toast';

const AuthContext = createContext(null),

AUTH_STORAGE_KEYS = { token: 'token', user: 'bloodconnect_user',
donorProfile: 'bloodconnect_donor_profile' };

```

```

const readStoredJson = key => { try { return
JSON.parse(localStorage.getItem(key)) || null; } catch { return
null; } };

const persistAuthSnapshot = ({ token, user, donorProfile }) => {

  token && localStorage.setItem(AUTH_STORAGE_KEYS.token, token);

  user && localStorage.setItem(AUTH_STORAGE_KEYS.user,
JSON.stringify(user));

  donorProfile ?
localStorage.setItem(AUTH_STORAGE_KEYS.donorProfile,
JSON.stringify(donorProfile))

  :
localStorage.removeItem(AUTH_STORAGE_KEYS.donorProfile);

};

const clearAuthSnapshot = () =>
Object.values(AUTH_STORAGE_KEYS).forEach(key =>
localStorage.removeItem(key));

export const useAuth = () => {

  const context = useContext(AuthContext);

  if (!context) throw new Error('useAuth must be used within an
AuthProvider');

  return context;

};

export const AuthProvider = ({ children }) => {

  const [user, setUser] = useState(() =>
readStoredJson(AUTH_STORAGE_KEYS.user)),

    [donorProfile, setDonorProfile] = useState(() =>
readStoredJson(AUTH_STORAGE_KEYS.donorProfile)),

    [loading, setLoading] = useState(true),

```

```

        [token, setToken] =
useState(localStorage.getItem(AUTH_STORAGE_KEYS.token));

    const syncAuthState = useCallback(async (authToken, fallbackUser
= null) => {

        try {

            api.defaults.headers.common.Authorization = `Bearer
${authToken}`;

            const { data } = await api.get('/auth/me', { params: { _ts:
Date.now() }, headers: { 'Cache-Control': 'no-cache' } });

            const currentUser = data?.user || fallbackUser, currentDonor
= data?.donorProfile || null;

            setUser(currentUser); setDonorProfile(currentDonor);

            persistAuthSnapshot({ token: authToken, user: currentUser,
donorProfile: currentDonor });

            return currentUser;

        } catch {

            if (fallbackUser) {

                setUser(fallbackUser);

                persistAuthSnapshot({ token: authToken, user:
fallbackUser, donorProfile: null });

            }

            return fallbackUser;

        }

    }, []);

    useEffect(() => {

        (async () => {

```

```

    const storedToken =
localStorage.getItem(AUTH_STORAGE_KEYS.token),

    storedUser = readStoredJson(AUTH_STORAGE_KEYS.user),

    storedDonor =
readStoredJson(AUTH_STORAGE_KEYS.donorProfile);

    if (storedToken) {

        setToken(storedToken);

        api.defaults.headers.common.Authorization = `Bearer
${storedToken}`;

        storedUser && (setUser(storedUser),
setDonorProfile(storedDonor));

        try { await syncAuthState(storedToken, storedUser); }

        catch {

            if (!storedUser) {

                clearAuthSnapshot();

                delete api.defaults.headers.common.Authorization;

                setUser(null); setDonorProfile(null); setToken(null);

            }

        }

        setLoading(false);

    }) ();

}, [syncAuthState]);

```

```

const login = useCallback(async (email, password) => {

    try {

```

```

        const { data } = await api.post('/auth/login', { email:
email.trim().toLowerCase(), password });

        api.defaults.headers.common.Authorization = `Bearer
${data.token}`;

        setToken(data.token);

        persistAuthSnapshot({ token: data.token, user: data.user,
donorProfile: null });

        const syncedUser = await syncAuthState(data.token,
data.user);

        toast.success(`Welcome back, ${syncedUser?.firstName ||
data.user.firstName}!`);

        return { success: true, user: syncedUser || data.user };

    } catch (e) {

        const message = e.response?.data?.message || 'Login failed';

        toast.error(message);

        return { success: false, message };

    }

}, [syncAuthState]);

const register = useCallback(async userData => {

    try {

        const { data } = await api.post('/auth/register', {
...userData, email: userData.email.trim().toLowerCase() });

        api.defaults.headers.common.Authorization = `Bearer
${data.token}`;

        setToken(data.token);

        persistAuthSnapshot({ token: data.token, user: data.user,
donorProfile: null });

```

```

        const syncedUser = await syncAuthState(data.token,
data.user);

        toast.success('Registration successful! Welcome to
BloodConnect.');
```

```

        return { success: true, user: syncedUser || data.user };

    } catch (e) {

        const message = e.response?.data?.message || 'Registration
failed';

        toast.error(message);

        return { success: false, message };

    }

}, [syncAuthState]);
```

```

const logout = useCallback(async () => {

    try { await api.post('/auth/logout'); }

    finally {

        clearAuthSnapshot();

        delete api.defaults.headers.common.Authorization;

        setUser(null); setDonorProfile(null); setToken(null);

        toast.success('Logged out successfully');

    }

}, []);
```

```

const updateProfile = useCallback(async profileData => {

    try {

        const { data } = await api.put('/auth/update-profile',
profileData);
```



```

        setUser(data.user);

        persistAuthSnapshot({ token, user: data.user, donorProfile
    });

    toast.success('Profile updated successfully');

    return { success: true, user: data.user };

    } catch (e) {

        const message = e.response?.data?.message || 'Update
failed';

        toast.error(message);

        return { success: false, message };

    }

}, [token, donorProfile]);

```

```

const changePassword = useCallback(async (currentPassword,
newPassword) => {

    try {

        await api.put('/auth/change-password', { currentPassword,
newPassword });

        toast.success('Password changed successfully');

        return { success: true };

    } catch (e) {

        const message = e.response?.data?.message || 'Password
change failed';

        toast.error(message);

        return { success: false, message };

    }

}, []);

```

```

const forgotPassword = useCallback(async email => {

  try {

    await api.post('/auth/forgot-password', { email });

    toast.success('Password reset email sent');

    return { success: true };

  } catch (e) {

    const message = e.response?.data?.message || 'Request
failed';

    toast.error(message);

    return { success: false, message };

  }

}, []);

const refreshUser = useCallback(async () => {

  try {

    const { data } = await api.get('/auth/me', { params: { _ts:
Date.now() }, headers: { 'Cache-Control': 'no-cache' } });

    if (data?.user) {

      setUser(data.user);

      setDonorProfile(data.donorProfile || null);

      persistAuthSnapshot({ token, user: data.user,
donorProfile: data.donorProfile || null });

    }

    return data;

  } catch {

    return null;

  }

}

```

```

    }, [token]));

    return (

      <AuthContext.Provider value={{

        user, donorProfile, token, loading,

        isAuthenticated: !!user,

        login, register, logout,

        updateProfile, changePassword,

        forgotPassword, refreshUser,

        setDonorProfile

      }}>

        {children}

      </AuthContext.Provider>

    );

  };

export default AuthContext;

*/

```

5.2.3 Code for Loads environment variables from .env file into application configuration

```

/*
require('dotenv').config();
const
express=require('express'),cors=require('cors'),helmet=require('helmet'),morgan=require('morgan'),compression=require('compression'),http=require('http'),path=require('path'),fs=require('fs');
const connectDB=require('./config/database');
const
{createCorsOriginValidator,getAllowedOrigins}=require('./config/origins');
const
{authRoutes,donorRoutes,requestRoutes,hospitalRoutes,adminRoutes,notificationRoutes,scheduleRoutes,statsRoutes}=require('./routes');

```

```

const
{errorHandler,notFound}=require('./middleware/errorHandler');
const
{apiRateLimiter,authRateLimiter,sanitizeRequest,preventHttpParamet
erPollution}=require('./middleware/security');
const {initializeSocket}=require('./socket/socketHandler');
const {startScheduledJobs}=require('./services/scheduledJobs');

const
app=express(),server=http.createServer(app),allowedOrigins=getAllo
wedOrigins(),PORT=process.env.PORT||5000;
app.set('trust proxy',1);
app.set('etag',false);

const io=initializeSocket(server);

app.use((req,res,next)=>{req.io=io;next();});
app.use(helmet({crossOriginResourcePolicy:{policy:'cross-
origin'}}));
app.use(cors({
  origin:createCorsOriginValidator(),
  credentials:true,
  methods:['GET','POST','PUT','DELETE','PATCH','OPTIONS'],
  allowedHeaders:['Content-Type','Authorization']
}));
app.use(express.json({limit:'10mb'}));
app.use(express.urlencoded({extended:true,limit:'10mb'}));
app.use(sanitizeRequest);
app.use(preventHttpParameterPollution(['sortBy','sortOrder','field
s']));
app.use(compression());
app.use(morgan(process.env.NODE_ENV==='development'? 'dev': 'combine
d'));
app.use('/uploads',express.static(path.join(__dirname,'uploads')))
;

app.get('/api/health',(req,res)=>res.json({
  status:'ok',
  timestamp:new Date().toISOString(),
  uptime:process.uptime(),
  environment:process.env.NODE_ENV
}));

app.use('/api',(req,res,next)=>{
  res.setHeader('Cache-Control','no-store');
  res.setHeader('Pragma','no-cache');
  next();
});

app.use('/api',apiRateLimiter);
app.use('/api/auth',authRateLimiter,authRoutes);
app.use('/api/donors',donorRoutes);
app.use('/api/requests',requestRoutes);
app.use('/api/hospitals',hospitalRoutes);
app.use('/api/admin',adminRoutes);
app.use('/api/notifications',notificationRoutes);
app.use('/api/schedules',scheduleRoutes);
app.use('/api/stats',statsRoutes);

if(process.env.NODE_ENV==='production'){
  const clientBuildPath=path.join(__dirname,'../client/build');

```

```

        if(fs.existsSync(clientBuildPath)){
            app.use(express.static(clientBuildPath));
        }

        app.get('*', (req, res)=>res.sendFile(path.join(clientBuildPath, 'index.html')));
    }
}

app.use(notFound);
app.use(errorHandler);

const startServer=async()=>{
    try{
        await connectDB();
        startScheduledJobs();
        server.listen(PORT, ()=>{
            console.log(`BloodConnect Server Running on Port
${PORT}`);
            console.log(`Environment:
${process.env.NODE_ENV||'development'}`);
            console.log(`Allowed Origins: ${allowedOrigins.join(',')}
`)});
        });
    }catch(error){
        console.error('Failed to start server:',error);
        process.exit(1);
    }
};

process.on('unhandledRejection',err=>{
    console.error('Unhandled Promise Rejection:',err);
    server.close(()=>process.exit(1));
});

process.on('uncaughtException',err=>{
    console.error('Uncaught Exception:',err);
    process.exit(1);
});

startServer();

module.exports={app,server,io};

*/

```

APPENDIX B – SCREENSHOTS

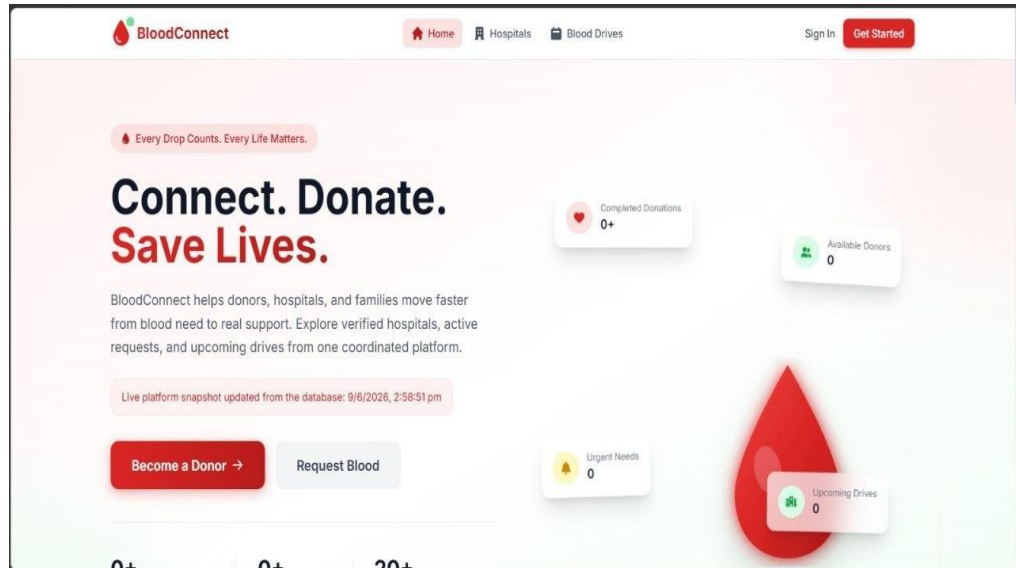


Fig B.1: Landing Page

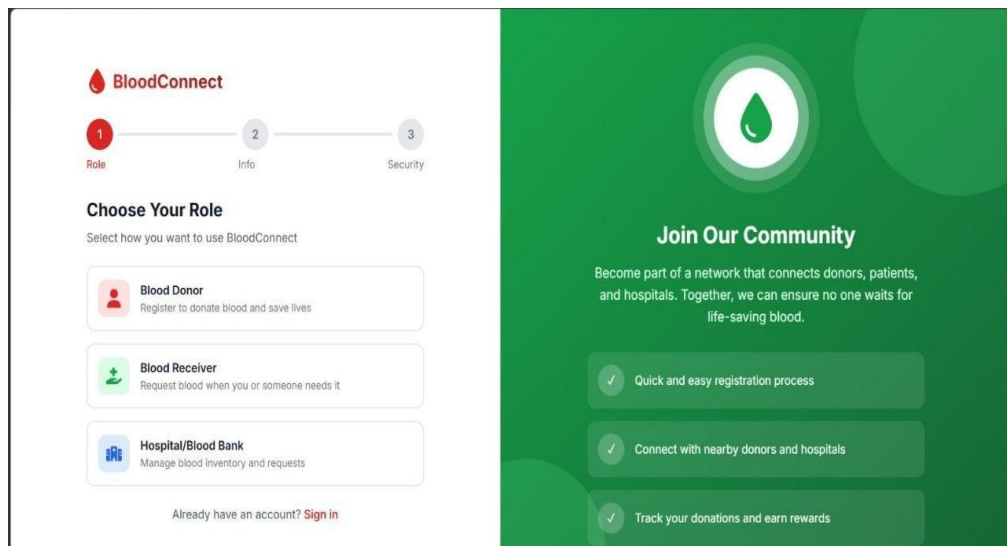


Fig B.2: Sign-Up Page

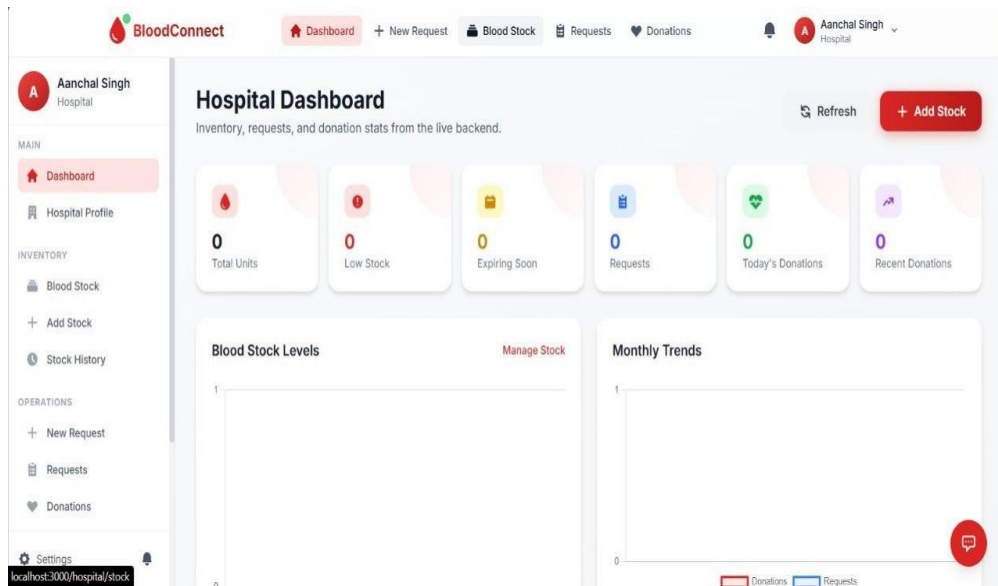


Fig B.3: Blood Donar Page

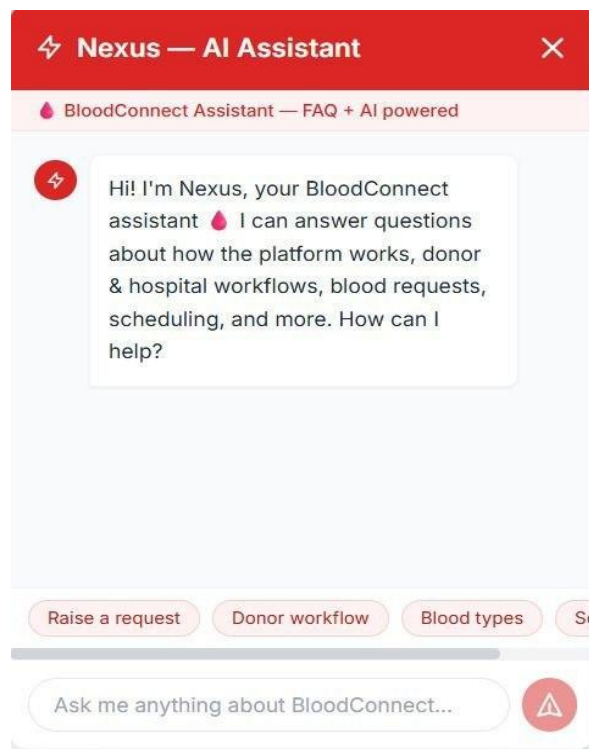


Fig B.4: AI Assistant

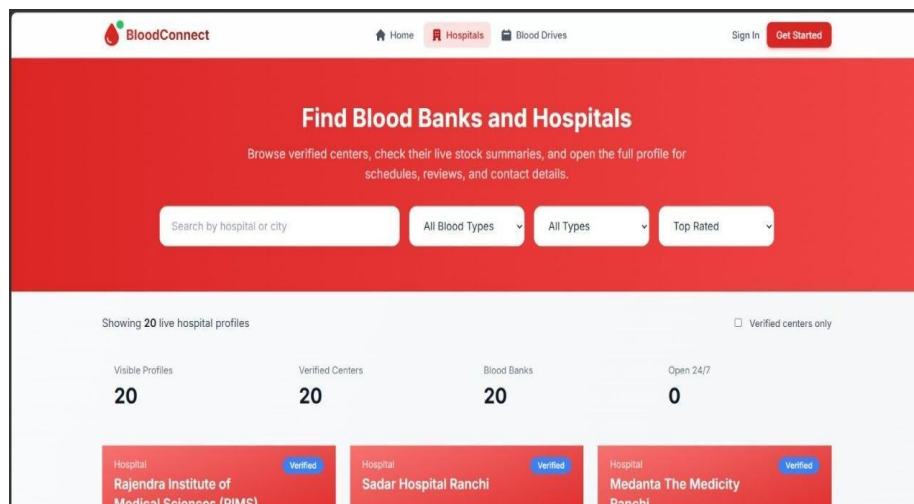


Fig B.5: Hospital/Blood Bank

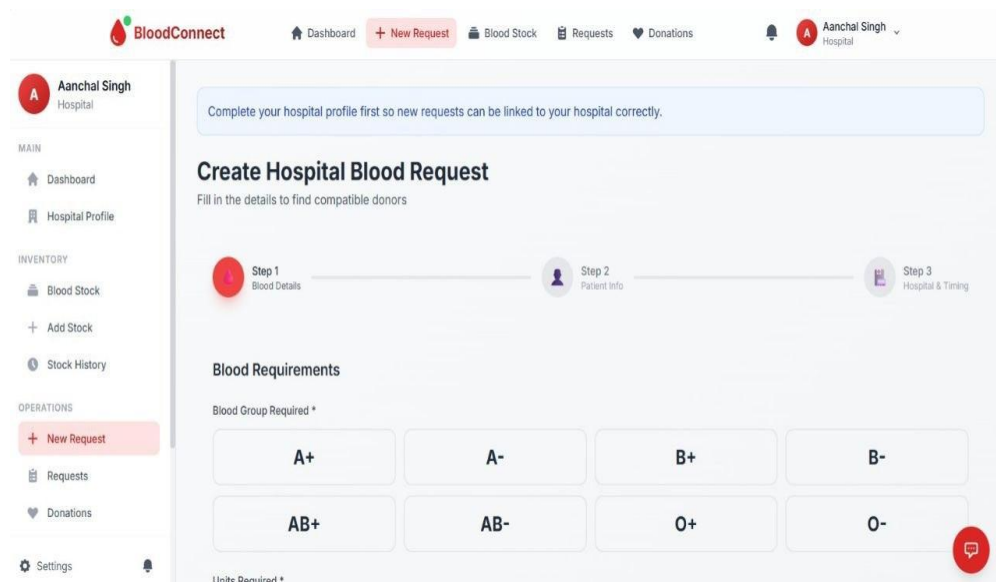


Fig B.6: New Request for Blood